# Number Theory

(CCS0047)

FEU ALABANG FEU DILIMAN FEU TECH

*Technology Driven by Innovation*

# WELCOME TO NUMBER THEORY!

German mathematician **Carl Friedrich Gauss** (1777–1855) said,
"Mathematics is the **queen of the sciences**—and number theory is the **queen of mathematics**."

# NOTABLE FIGURES

- **Euclid (300 BC)**: The Father of Geometry, but he also wrote the Euclidean Algorithm—the fastest way for a computer to find the Greatest Common Divisor (Module 2).
- **Eratosthenes (200 BC)**: Created a brilliant algorithm to filter out prime numbers called the **Sieve of Eratosthenes** (Module 1).
- **Pierre de Fermat (1600s)**: A lawyer who did math for fun. His "Little Theorem" is the backbone of modern algorithmic primality testing (Module 4).
- **Rivest, Shamir, & Adleman (1977)**: Three computer scientists who took 2,000-year-old number theory and used it to invent RSA Encryption — the protocol that currently secures the entire internet (Module 6).

# NUMBER THEORY

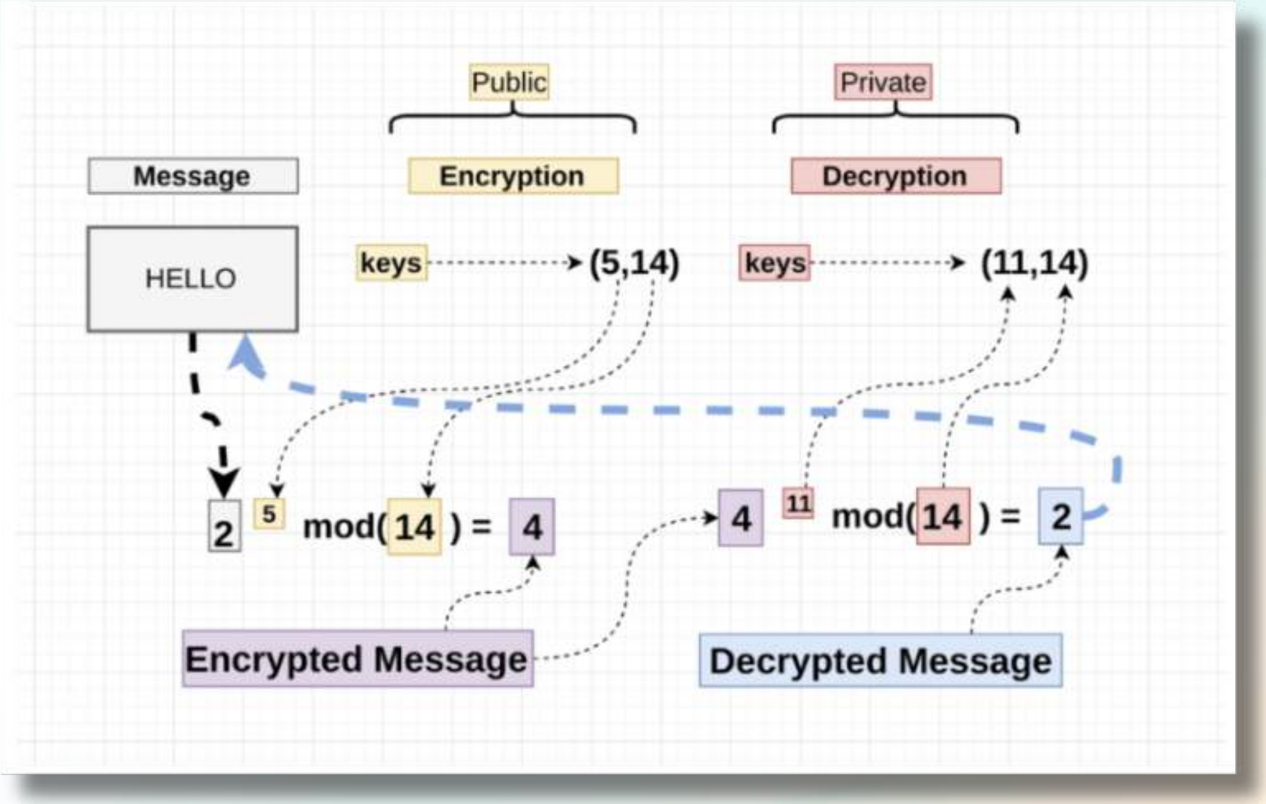
Number theorists study **prime numbers** as well as the properties of mathematical objects constructed from integers *(for example, rational numbers)*, or defined as generalizations of the integers *(for example, algebraic integers)*.

- Also called as **"Higher Arithmetic"**
- Essentially the study of mathematical interactions and number types
- Odds, evens, squares, integers, etc. prove theorems and answer questions about their relationships
- Use **formal mathematical proofs** to answer these questions

# NUMBER THEORY

In the 20th century, the concepts in Number Theory have a more evident application.

These are now used for **encryption schemes**. Several encryption methods use prime factorizations - factors of a huge number.



# Why Number Theory? (The Source Code of Math)

In programming, **computers do not have "common sense."** A computer does not intuitively know what "1 + 1" is.

**Axioms** are the source code of mathematics. Before we can write complex algorithms, we have to define the **absolute ground rules of logic from scratch**. We have to prove the "obvious" so we can program the "complex."

# Number Theory

(CCS0047)

## Module 1: Foundations of Discrete Math

Subtopic 1: Real Number System and Set Notations

FEU ALABANG FEU DILIMAN FEU TECH

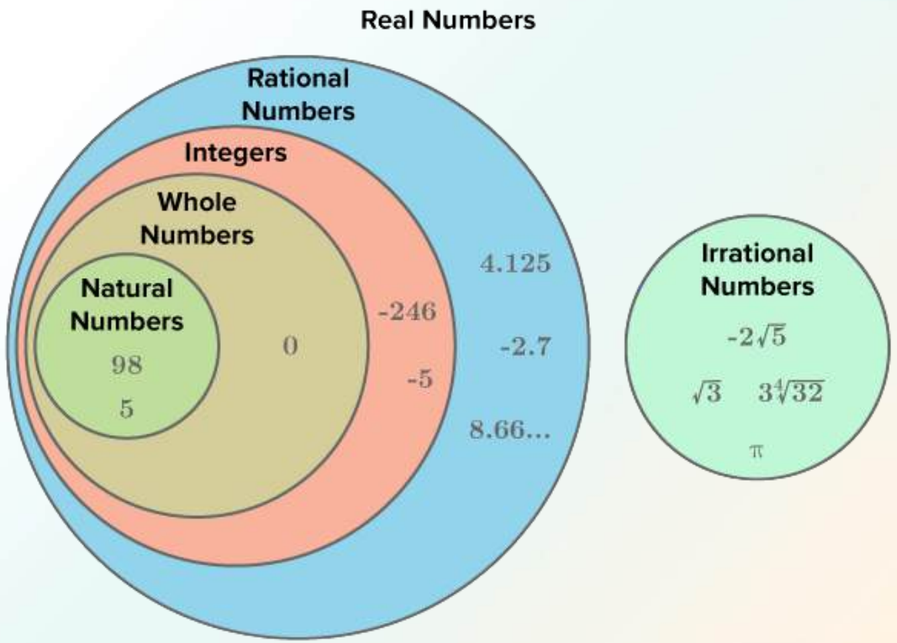
*Technology Driven by Innovation*

# Real Number System

This diagram shows how the numbers we use are organized. For this course, it's helpful to remember the main categories:

**Real Numbers**: The set of all numbers on the number line. They are divided into two main groups:

- **Rational Numbers**: Any number that can be written as a simple fraction, like 4.125 or −5.
- **Irrational Numbers**: Numbers that cannot be written as a simple fraction and have non-repeating decimals, like π or 3.

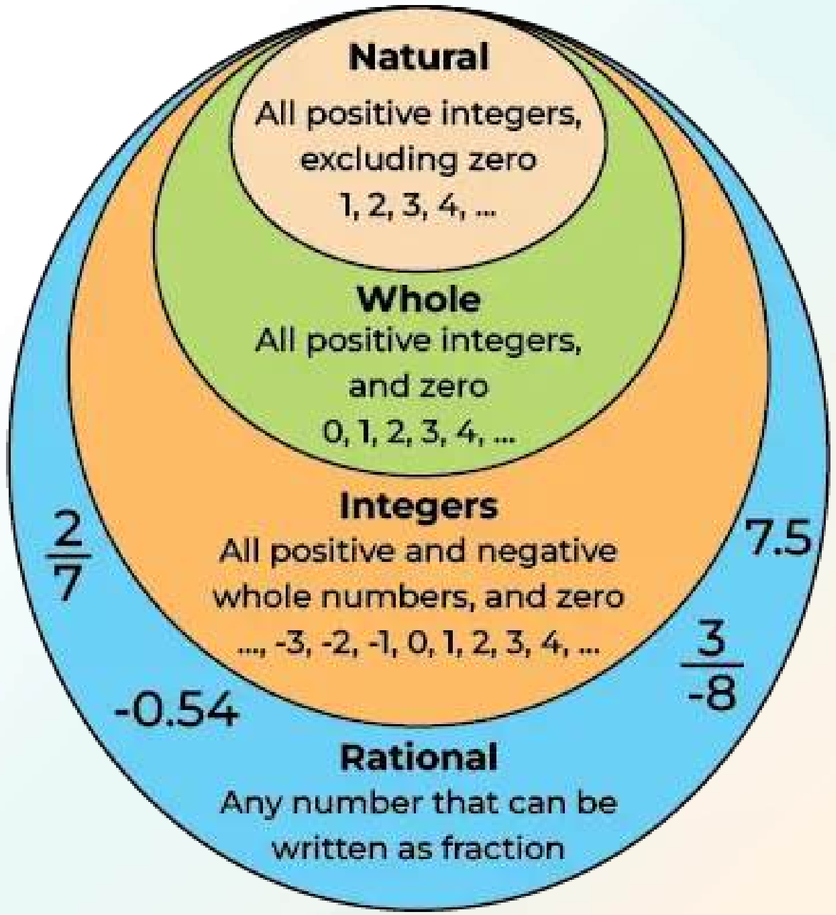


# Real Number System

Inside the Rational Numbers, we find nested groups:

- **Integers (Z):** All positive and negative whole numbers, including zero (e.g., -246, -5, 0, 98).
- **Whole Numbers:** The non-negative integers (0, 1, 2, ...).
- **Natural Numbers (N):** The positive "counting" numbers (1, 2, 3, ...).

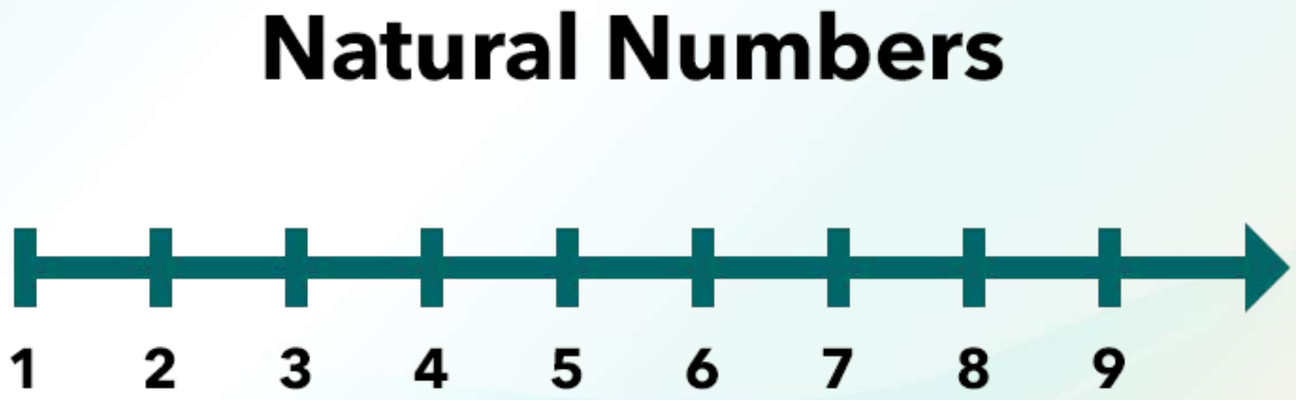
*NOTE: While all these numbers exist, our work in this course will primarily take place in the world of Integers (Z) and Natural Numbers (N).*



# Types (Sets) of Numbers

**Natural Numbers** *(counting numbers)*
The natural numbers are the part of the number system that includes all the positive integers from 1 through infinity. They are used for the purpose of counting.

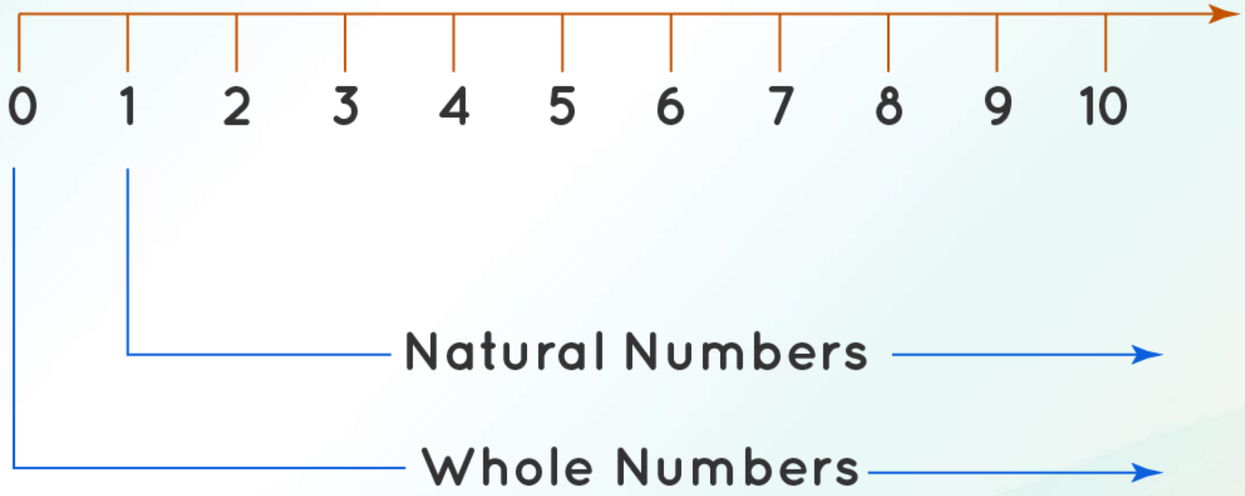
The set of natural numbers is usually represented by the letter "N".



# Types (Sets) of Numbers

**Whole Numbers**

The whole numbers are the numbers without fractions and it is a collection of positive integers and zero. It is represented by the symbol “W”. Zero as a whole represents nothing or a null value.



# Types (Sets) of Numbers

**Integers**

The integers are a set of numbers consisting of the natural numbers, their additive inverses, and zero. In other words, they include all positive numbers, negative numbers, and 0.

The set of integers is usually represented by the letter "Z". It can also be represented by the letter "J".

# Types (Sets) of Numbers

**Rational Numbers**
Numbers that can be written as a fraction or as a ratio.

**a divided by b** where a is a whole number and b is also a whole number.

# Types (Sets) of Numbers

## Irrational Numbers

An irrational number is a real number that cannot be expressed as a ratio of integers; for example, √2 is an irrational number. **We cannot express any irrational number in the form of a ratio**, such as p/q, where p and q are integers, q≠0.

3.14159265358979323846264338327950288419716939937510582097494459230781
64062862089986280348253421170679821480865132823066470938446095505822317253594081284811174502841027019385211055596446229489549303819644288109756659334461284756482337867831652712019091456485669234603486104543266482133936072602491412737245870066063155881748815209209628292540917153643678925903600113305305488204665213841469519415116094330572703657595919530921861173819326117931051185480744623799627495673518857527248912279381183119491298336733624406566430860213949463952247371907021798609437027705392171762931767523846748184676694051320005681271452635608277857713427577896091736371787214684409012249534301465495853710507922796892589235420199561121290219608640344181598136297747713099605187072113499999983729780499510597317328160963185950244594553469083026425223082533446850352619311881710100031378387528865875332083814206171776691473035982534904287554687311595628638823537875937519577818577805321712268066130019278766111959092164201989380952572010654858632788659361533818279682303019520353018529689957736225994138912497217752834791315155748572424541506959508295331168617278558890750983817546374649393192550604009277016711390098488240128583616035637076601047101819429555961989467678374494482553797747268471040475346462080466842590694912933136770289891521047521620569660240580381501935112533824300355876402474964732639141992726042699227967823547816360093417216412199245863150302861829745557067498385054945885869269956909272107975093029553211653449872027559602364806654991198818347977535663698074265425278625518184175746728909777727938000816470600161452491921732172147723501414419735685481613611573525521649252168509965502435448115935963467594767026465425670859382165626618661733193450342765773417317985439069689152523108619284881685962294445071164149655710937187107163907103823047184338385112936511212836845844805683702511456052762411071345163150862861534382686338686348441388601463824255966506835224388050686471603709016106119976535279494272916864794607854367227036942822383483448386768515047146103983425571860416336312038416055362964457285602274496473029362067192528803191911819767403685474238193684806602132928306227243029502689434865835506376998909362506017668119049676231224673338375031501024240432432523892596276766375027705471966975674713682323058989411404154474392599359963177309809709256688251672185276697027614503714736819325718591069183716483631229431325625763382475999046505941616113614836620696660571096350944434942424555079388377035822510762941065939875479276754529434825226363586767103809161319388116766256157154941311616651535062034434949946937764440760158286298517539019368226191937121238438971203188334611389670376047759095118674024262661790942207965289171474179691839051751328388352332592571590701165941941689687698293960306714079377030104751147484215968292326386794276778045602698720668822483265207603318003988953043503996351231571460413479189330939965694603053311796974154511273172123823968316286627386286592784658758671875051175462116962679430819382312594742058992792859894058955835630670121352656533461465568194799362233936739736686325050237689573516832926194194320827505943864565967034403053488067119767742465806372485309066368892963035630832792411529691325367255071234448655294231856072090648323861941725821284849779212965365698540856316204589325994945137342765403276626860256527569960995056350112533765961765091303652863629411754111960276519411110979416983481235201056510354052249317832967480762334485539864962572161323450778889596194761536577342962773548489748160608168348681703252353309531967473694571147111655462941741798283019566849852324720292037762475823483550303765136470751285892116038362757046084422038679325965620274596863205488702302319319852359599523809635130474822665433860285965948113640859471127659686461963621271941596754129159403692439320921960609273238758328395909813405177477167260793792255289411353751943766309627960451508162352582897256958312591143909015630352947137619510918417317641831431356588011286038283329338657752835527904421017671337781571313267068485422049531098817765610273179718706106967116094337925172559328489631960121876579471103697698179470757408745150423254685251275777168808457909569238894422007983823151833820754389892084754844091715135661127619466967374772417310113184697362718829620116663733809719301781766178577805893176108101442209219542416468520398651952193461580438671037457309968618114779541471970596029635445618126238390589687411069416589862092416491042424572417029516593018425392068059074054210891267056236655654398285859361065822648237116093353191829481837943776759444742403515818996915667743495402185592187193864844492310340181527501966428670772960863832046457501048687860864233334547762416862518983569485562099219222184272550254256887671790494601653466804988627232791786085784383827967976681454100953883786360950680064225125205117392984896084128488626945604241965285022210661186306744278622039194945047123713786960956364371917287467764657573962413890865832645995813390478027590099465764078951269468398352595709825822620518852966672232829720830502046117117626845041608960524301198553473963408650329180774672617457472756795393618206233689607120650289534869608

# Types (Sets) of Numbers

**Other number sets:**

- Even numbers - subset from the integers
- Odd numbers - numbers are not even
- Prime Numbers - a whole number greater than 1 that cannot be exactly divided by any whole number other than itself and 1 (e.g. 2, 3, 5, 7, 11).
- Composite Numbers - numbers that are not prime

# Important Notations for Sequences and Series

**Summation notation (or sigma notation)** allows us to write a long sum in a single expression.

Stop at $n = 3$

(inclusive)

$$\sum_{n=1}^{3} 2n - 1$$

Expression for each term in the sum

Start at $n = 1$

# Important Notations for Sequences and Series

**This is a summation of the expression 2n−1 for integer values of n from 1 to 3:**

$$\sum_{n=1}^{3} 2n - 1$$

= [2(1) − 1] + [2(2) − 1] + [2(3) − 1]

= 1 + 3 + 5

**= 9**

*n is our summation index. When we evaluate a summation expression, we keep substituting different values for our index.*

# Important Notations for Sequences and Series

**Product notation, or pi notation,** is a mathematical tool that indicates repeated multiplication.

*Stop at n = 4*

$$\prod_{k=1}^{4}(2k)$$

*Expression for each term in the product*

*Start at k = 1*

# Important Notations for Sequences and Series

**Product notation, or pi notation,** is a mathematical tool that indicates repeated multiplication.

$$\prod_{k=1}^{4}(2k) = (2 \cdot 1) \cdot (2 \cdot 2) \cdot (2 \cdot 3) \cdot (2 \cdot 4) = 2 \cdot 4 \cdot 6 \cdot 8 = \boxed{384}$$

Similar to how **∑ (Sigma)** is used for adding terms, the **Product Notation (∏)** is a shorthand for multiplying a sequence of terms together.

# Important Notations for Sequences and Series

**Factorial notation** is a mathematical symbol that represents the product of all positive integers from 1 up to a given number. It is written as an exclamation mark (!) after a positive integer n and is read as "n factorial".

**Examples of factorial notation:**

- $4! = 4 \times 3 \times 2 \times 1 = \boxed{24}$
- $5! = 5 \times 4 \times 3 \times 2 \times 1 = \boxed{120}$
- $6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = \boxed{720}$

**Recursive Property**: A factorial can also be defined recursively as **n! = n x (n - 1)!**

# Important Notations for Sequences and Series

Now that we have our toolkit (Summations, Factorials), how do we actually prove that our formulas work every time? We need to start at the very beginning... **with Axioms**.

# Number Theory

(CCS0047)

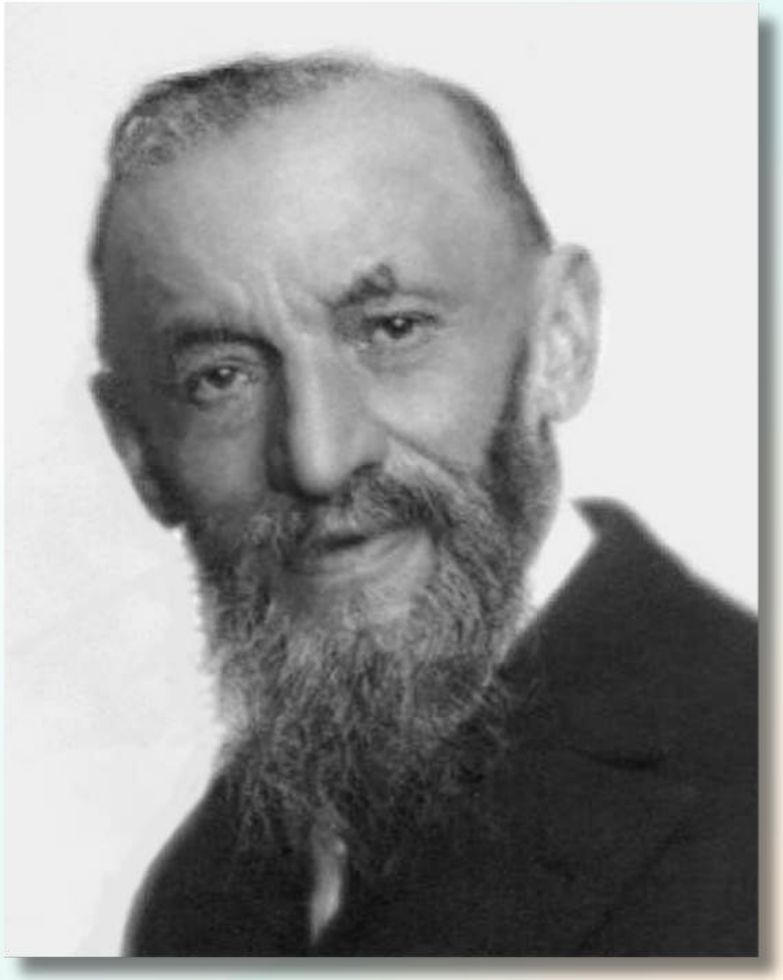
## Module 1: Foundations of Discrete Math

Subtopic 2: Axiomatic Logic and Proof by Induction

# PEANO AXIOMS

The Goal: He wanted to define the **natural numbers** (N) using the smallest possible set of rules. He didn't want to just "assume" numbers existed; he wanted to build them from scratch using logic.

Peano originally started his axioms with 1.



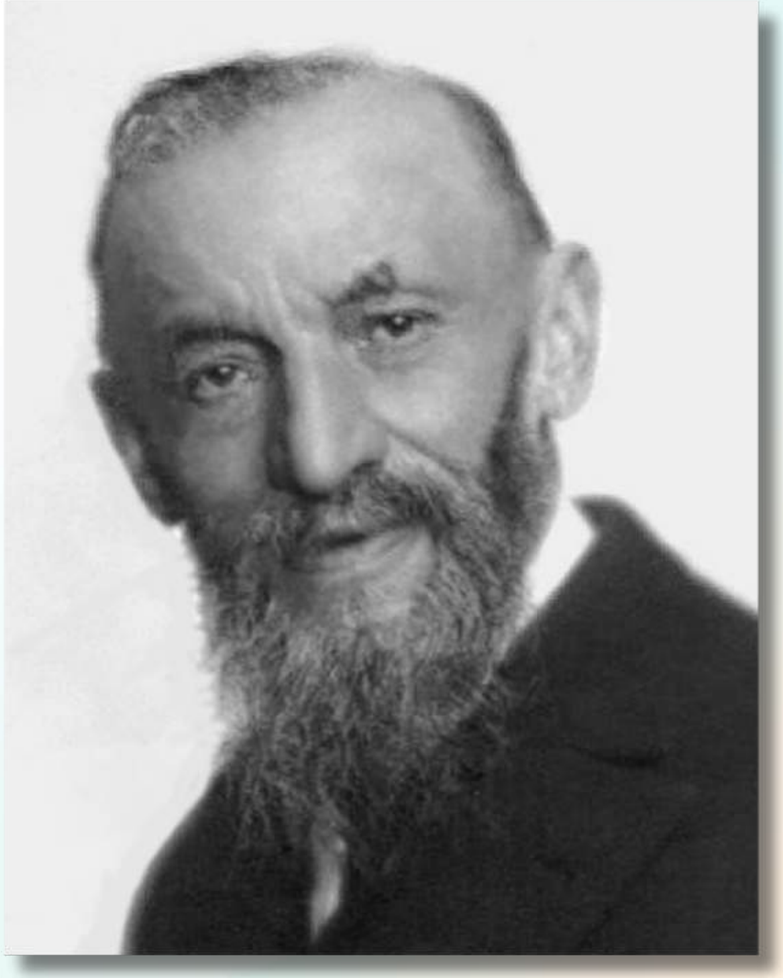
Giuseppe Peano (1889)

# PEANO AXIOMS

Most modern mathematicians and Computer Scientists **now start the axioms with 0**.

In Set Theory, **0 represents the "Empty Set"** (the start of everything).

In Computer Science, we start at 0 for **Array Indexing** and **Memory Addressing**.



Giuseppe Peano (1889)

# PEANO AXIOMS

It is important to keep in mind that when Peano and others constructed these axioms, their goal was to provide the fewest axioms that would generate the natural numbers that everyone was familiar with.

The insight is that this could be done by asserting the existence of **at least one natural number**, and then defining a function, called **successor function**, that can be used to construct the remaining natural numbers.

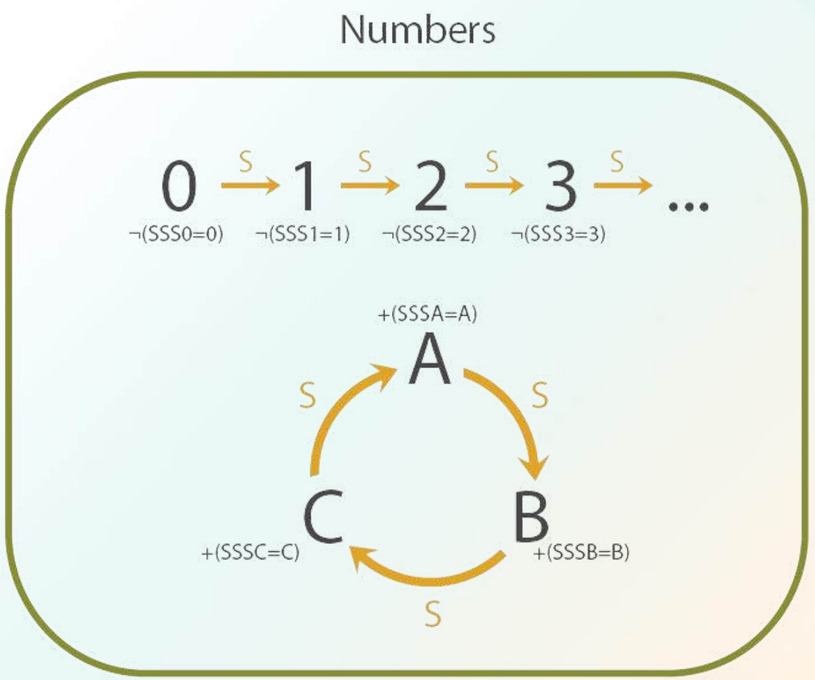
An obvious element to **axiomatically include in the natural numbers is zero.**

# PEANO AXIOMS

The Peano axioms typically consist of **five axioms**:

1. Zero is a natural number.

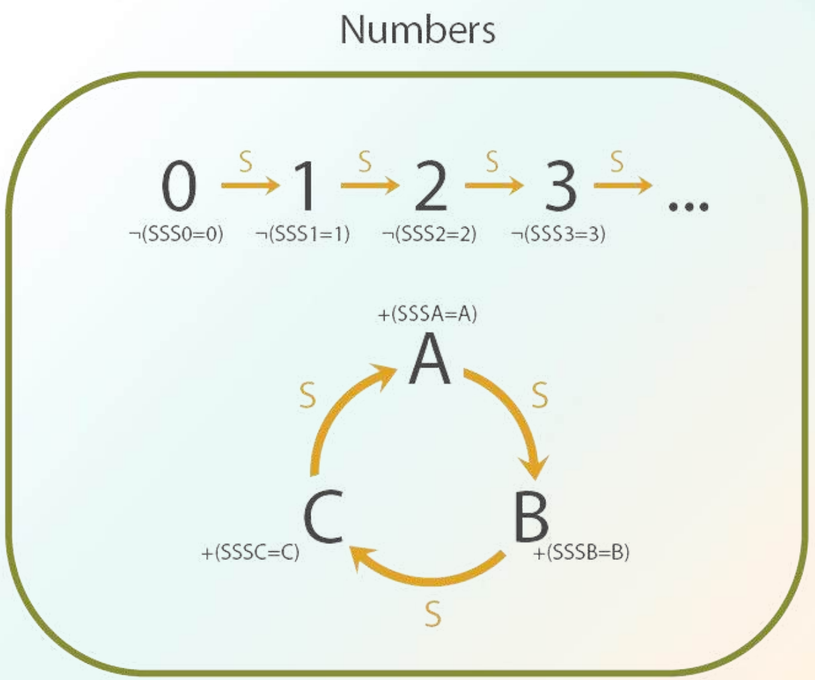
2. **Successor function**: Every natural number n has a unique successor denoted by S(n). This axiom asserts the existence of a one-to-one correspondence between the natural numbers and their successors.



# PEANO AXIOMS

3. **No two numbers have the same successor**: For any two natural numbers m and n, if S (m) = S(n), then m = n. This axiom ensures that each natural number has a distinct successor.

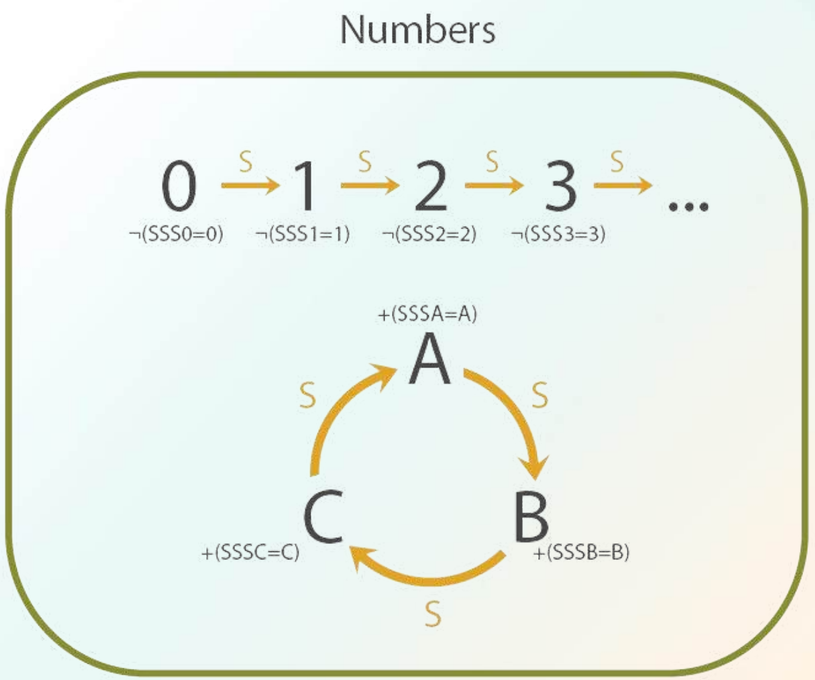
4. **Principle of mathematical induction**: If a property holds for 0 and whenever it holds for k it also holds for **S(k)**, then it holds for all natural numbers. This axiom allows for proofs by **mathematical induction**.



# PEANO AXIOMS

5. **Axiom of infinity**: There exists a natural number called ω (omega), which is not the successor of any natural number. This axiom asserts the existence of an infinite set of natural numbers.

These axioms provide a formal **framework for reasoning about the natural numbers** and allow for the development of arithmetic operations, such as addition and multiplication, and various properties of the natural numbers.



# SUCCESSOR OPERATION

The **successor function**, often denoted by $S$ is a fundamental concept in the Peano axioms and in the theory of natural numbers. It is a unary function that takes a natural number as its input and returns the next natural number in the sequence.

Formally, if $n$ is a natural number, then $S(n)$ represents its successor. For example:

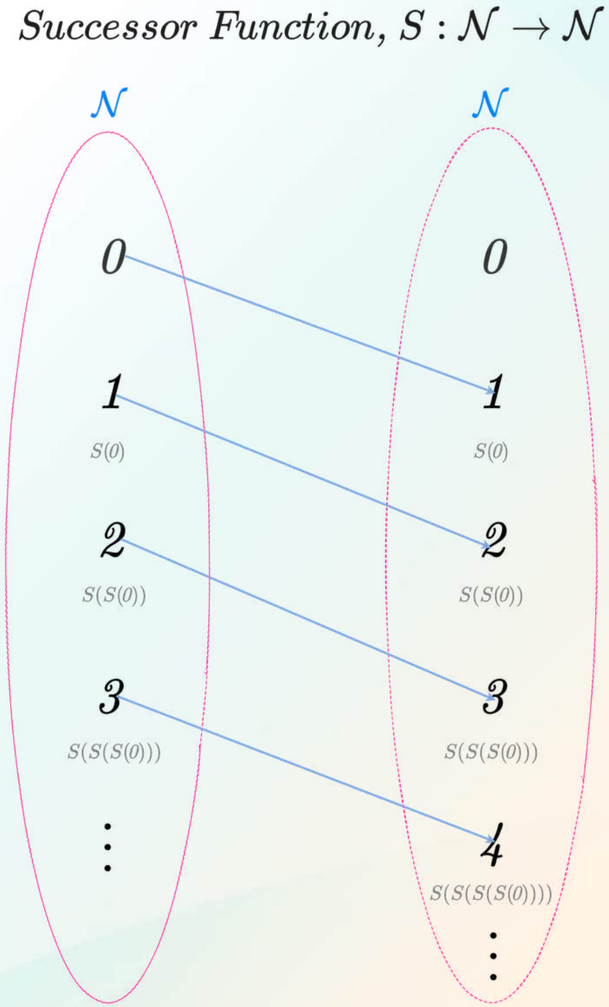
- If n = 0, then S(n) = 1
- If n = 1, then S(n) = 2
- If n = 2, then S(n) = 3, and so on.

# SUCCESSOR OPERATION

Addition on natural numbers is defined recursively using the **Successor Function** $S(n)$, which represents the "next" number (e.g., $S(0) = 1$, $S(1) = 2$) ().

**Recursive Definition**:

1. **Base Case**: m + 0 = m ()
2. **Recursive Rule**: m + S(n) = S(m) + n ()



# SUCCESSOR OPERATION

## Successor Operation Trace: 5 + 2

| Mathematical Step | Explanation / Rule Applied |
|---|---|
| **5 + 2 = 5 + S(1)** | We rewrite 2 using the successor function as **S(1)**. |
| **S(5) + 1** | We apply the recursive rule: **m + S(n) = S(m) + n**. |
| **6 + 1** | We evaluate **S(5)** to get its next natural number, which is **6**. |
| **6 + S(0)** | We rewrite 1 using the successor function as **S(0)**. |
| **S(6) + 0** | We apply the recursive rule again: **m + S(n) = S(m) + n**. |
| **7 + 0** | We evaluate **S(6)** to get its next natural number, which is **7**. |
| **7** | We apply the base case rule: **m + 0 = m** to find the final result. |

**YOUR TURN! Try: 6 + 3**

# SUCCESSOR OPERATION

**For: 6 + 3**

= 6 + S(2)
= S(6) + 2
= 7 + 2
= 7 + S(1)
= S(7) + 1
= 8 + 1
= 8 + S(0)
= S(8) + 0
= 9 + 0
= 9

*This formal process allows us to generate natural numbers from the fewest possible rules.*

*Including zero as an obvious element provides the necessary base case for these recursive operations.*

*This trace visually demonstrates how the successor function "shifts" the value from the right side of the equation to the left until the right side becomes zero*

# THE WELL-ORDERING PRINCIPLE

The **Well-Ordering Principle (WOP)** is a fundamental concept in set theory and mathematical logic. It states that every non-empty set of natural numbers (or equivalently, every non-empty subset of the natural numbers) contains a least element.

This **principle essentially guarantees that there is no infinite decreasing sequence of natural numbers**. You will always eventually hit a "bottom" or a starting point.

**Examples:**

- A Finite Set: Let S = {15, 8, 42, 99}. **The least element is 8**.
- An Infinite Set: Let E be the set of all positive even numbers {2, 4, 6, 8, ...}. **The least element is 2**.
- The Entire Set of Natural Numbers: {0, 1, 2, 3, ...}. **The least element is 0**.

# THE WELL-ORDERING PRINCIPLE

Consider the set A = {2, 4, 6, 8, 10, ….}, which consists of all even natural numbers greater than 0.

Now, let's show that this set satisfies the **Well-Ordering Property**:

1. **Non-emptiness:** The set $A$ is non-empty because it contains at least one element, namely 2.
2. **Existence of least element:** Every non-empty subset of $A$ contains a least element. For example, consider the subset B = {6, 8, 10, 12, …}. **The least element is 6**.

# THE WELL-ORDERING PRINCIPLE

The example in the previous slide demonstrates the **Well-Ordering Property** because it shows that even though the set $A$ is infinite, every non-empty subset of $A$ has a smallest element.

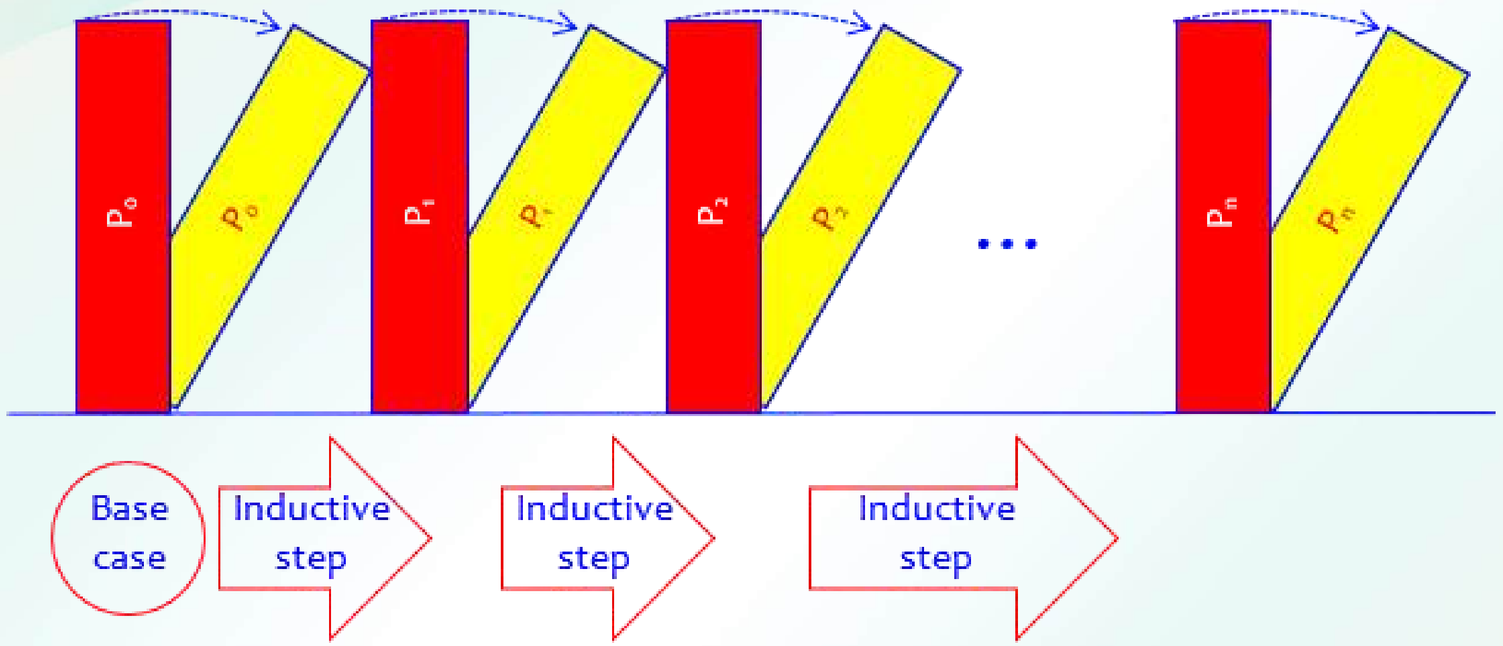
This property is essential for various mathematical proofs, particularly in the context of **induction** and establishing existence results.

# PROOF BY INDUCTION

**Mathematical induction** is a powerful and widely used method of mathematical proof. It's primarily used to prove statements about natural numbers (integers greater than or equal to zero), although it can sometimes be adapted to other contexts.

It is also a fundamental proof technique used in various areas of mathematics, including **number theory, combinatorics, algebra,** and **analysis**. It's particularly useful for proving properties of sequences, series, and recursive definitions. It's a cornerstone of mathematical reasoning and plays a crucial role in many mathematical discoveries and developments.

# PROOF BY INDUCTION




**Mathematical induction** is a technique often used to prove statements about **sequences** or **patterns**, where one first demonstrates the statement holds for a **base case** and then shows that if it holds for any given value, it also holds for the next, thereby proving its validity for all values.

# PROOF BY INDUCTION

The primary purpose of mathematical induction is to prove that a statement holds for all natural numbers (or some subset of natural numbers). This is typically done by **following two main steps**:

**Base Case:** The first step is to prove that the statement holds true for the smallest value in the domain (usually n = 0 or n = 1 depending on the context). This is often referred to as the base case.

**Inductive Step:** The second step is to prove that assuming the statement holds for some arbitrary natural number k (this is called the induction hypothesis), then the statement also holds for the next natural number, k+1.

# PROOF BY INDUCTION

**Claim**: $1 + 3 + 5 + \ldots + (2n-1) = n^2$

We start with the base case. This is usually 0 or 1 if not specified. Start with some examples below to make sure you believe the claim.

# PROOF BY INDUCTION

**Claim**: $1 + 3 + 5 + \ldots + (2n-1) = n^2$

**Step 1: Result is true for n = 1**

That is $2(1)-1 = (1)^2$

$1 = 1$ (True)

**Step 2: Assume that result is true for n = k**

$1 + 3 + 5 + \ldots + (2k-1) = k^2$

**Step 3: Check for n = k + 1**

$1 + 3 + 5 + \ldots + (2\mathbf{(k+1)}-1) = \mathbf{(k+1)^2}$

**WHY? Assume the equality**

i.e. $1 + 3 + 5 + \ldots + (2k-1) = k^2$

assuming this

is equal to this

**Step 4: Simplify the equation**

$\mathbf{k^2} + (2(k+1)-1) = (k+1)^2$

**Step 5: Perform the operation**

$k^2 + 2k + 2 - 1 = (k+1)^2$

$k^2 + 2k + 1 = (k+1)^2$

$\mathbf{(k+1)^2 = (k+1)^2}$

# PROOF BY INDUCTION

**Prove that 2+4+6+.... + 2n = n (n + 1)**

**Step 1: Result is true for n = 1**

$$2n = n(n + 1)$$
$$2(1) = (1)(1 + 1)$$
$$2 = 2$$

**Step 2: Assume that result is true for n = k**

$$2+4+6+.... + 2k = k(k + 1)$$

**Step 3: Check for n = k + 1**

$$2+4+6+.... + 2(k+1) = (k + 1)((k + 1) + 1)$$

**Step 4: Simplify the equation**

$$(k(k + 1)) + 2(k+1) = (k+1)(k+2)$$

**Step 5: Perform the operation**

$$(k(k + 1)) + 2(k+1) = (k+1)(k+2)$$
$$k^2 + k + 2k + 2 = k^2 + k + 2k + 2$$
$$k^2 + 3k + 2 = k^2 + 3k + 2$$

# SEATWORK

## Solutions to the following:

(a) $1 + 2 + 3 + \ldots + n = \dfrac{n(n+1)}{2}$ for all $n \geq 1$

(b) $1 + 3 + 5 + \ldots + (2n-1) = n^2$ for all $n \geq 1$

# SEATWORK

## Solutions to the following:

Prove that $1^3+2^3+3^3+\cdots+n^3$ is equal to $\left(\frac{n(n+1)}{2}\right)^2$ for all positive integers n.

Prove that $1+2+3+\ldots+n = \left(\frac{n(n+1)}{2}\right)$ Prove this using Mathematical Induction.

# Number Theory

(CCS0047)

**Module 1: Foundations of Discrete Math**

Subtopic 3: Prime Numbers

# Prime Numbers

**Prime numbers** are used in **cryptography** due to their role in creating mathematically hard problems, such as integer factorization and discrete logarithms, forming the basis for secure algorithms like **RSA** and **Diffie-Hellman**.

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 2 | 3 | 5 | 7 | 11 | 13 | 17 | 19 | 23 | 29 |
| 31 | 37 | 41 | 43 | 47 | 53 | 59 | 61 | 67 | 71 |
| 73 | 79 | 83 | 89 | 97 | 101 | 103 | 107 | 109 | 113 |
| 127 | 131 | 137 | 139 | 149 | 151 | 157 | 163 | 167 | 173 |
| 179 | 181 | 191 | 193 | 197 | 199 | 211 | 223 | 227 | 229 |
| 233 | 239 | 241 | 251 | 257 | 263 | 269 | 271 | 277 | 281 |
| 283 | 293 | 307 | 311 | 313 | 317 | 331 | 337 | 347 | 349 |
| 353 | 359 | 367 | 373 | 379 | 383 | 389 | 397 | 401 | 409 |
| 419 | 421 | 431 | 433 | 439 | 443 | 449 | 457 | 461 | 463 |
| 467 | 479 | 487 | 491 | 499 | | | | | |

# Prime Numbers

**The Formal Definition:**
An integer **p > 1** is called a **prime number** if its only **positive divisors** are **1** and **p**.

**In Formal Logic:**
$p \text{ is prime} \Leftrightarrow \forall\ d \in \mathbb{Z}^{+}, (d \mid p \Rightarrow d = 1 \lor d = p)$

**Composite Numbers**: Must have more than two divisors.

*Note: The **number 1** Has only one positive divisor (itself). Therefore, it fails the definition for both.*

# Composite Numbers

An **integer n > 1** is called **composite** if it is **not prime**.

- In other words, a composite number has at least one divisor other than 1 and itself.
- **Examples:** 4, 6, 8, 9, 10, 12...
- Every composite number can be broken down into prime factors **(Fundamental Theorem of Arithmetic)**.

# Prime Number Generation

**The Sieve of Eratosthenes: Ancient Algorithm, Modern Code**

**What is a "Sieve"?**
A sieve is a tool used to separate wanted elements from unwanted material. In Number Theory, we use it to filter out composite numbers, leaving only the primes behind.

**The Floor Function:** $\lfloor x \rfloor$ represents the greatest integer less than or equal to x.

# Prime Number Generation

**The Sieve of Eratosthenes: Ancient Algorithm, Modern Code**

**The Algorithmic Strategy:**
Instead of testing if each number is prime (which is slow), we assume all numbers are prime. Then, we methodically cross out the multiples of known primes.

**The Steps:**

1. Create a list of consecutive integers from **2 to n**.
2. Let **p = 2** (the first prime number).
3. Cross out all multiples of p (2p, 3p, 4p...) as **they are composite**.
4. Find the next uncrossed number, assign it to p, and repeat.

# Prime Number Generation

**Algorithm Trace:** Sieve of n = 20
**Goal:** Find all primes ≤ 20.
**Step 1:** Start with an array of numbers 2 through 20.

| | | | | |
|---|---|---|---|---|
| 2 | 3 | 4 | 5 | 6 |
| 7 | 8 | 9 | 10 | 11 |
| 12 | 13 | 14 | 15 | 16 |
| 17 | 18 | 19 | 20 | |

# Prime Number Generation

**Algorithm Trace:** Sieve of n = 20
**Step 2:** p = 2. Keep 2, cross out its multiples (4, 6, 8...)

| | | | | |
|---|---|---|---|---|
| 2 | 3 | ~~4~~ | 5 | ~~6~~ |
| 7 | ~~8~~ | 9 | ~~10~~ | 11 |
| ~~12~~ | 13 | ~~14~~ | 15 | ~~16~~ |
| 17 | ~~18~~ | 19 | ~~20~~ | |

# Prime Number Generation

**Algorithm Trace:** Sieve of n = 20
**Step 3:** Move to next uncrossed number (p = 3).
Cross out multiples (9, 15...)

| | | | | |
|---|---|---|---|---|
| 2 | 3 | ~~4~~ | 5 | ~~6~~ |
| 7 | ~~8~~ | ~~9~~ | ~~10~~ | 11 |
| ~~12~~ | 13 | ~~14~~ | ~~15~~ | ~~16~~ |
| 17 | ~~18~~ | 19 | ~~20~~ | |

# Prime Number Generation

**Algorithm Trace:** Sieve of n = 20
**Step 4:** Move to next uncrossed number (p = 5).
Cross out multiples. **(All are crossed out.)**

| | | | | |
|---|---|---|---|---|
| 2 | 3 | ~~4~~ | 5 | ~~6~~ |
| 7 | ~~8~~ | ~~9~~ | ~~10~~ | 11 |
| ~~12~~ | 13 | ~~14~~ | ~~15~~ | ~~16~~ |
| 17 | ~~18~~ | 19 | ~~20~~ | |

# Prime Number Generation

**Algorithm Trace:** Sieve of n = 20

**The remaining numbers are 2, 3, 5, 7, 11, 13, 17, 19)**

| | | | | |
|---|---|---|---|---|
| 2 | 3 | ~~4~~ | 5 | ~~6~~ |
| 7 | ~~8~~ | ~~9~~ | ~~10~~ | 11 |
| ~~12~~ | 13 | ~~14~~ | ~~15~~ | ~~16~~ |
| 17 | ~~18~~ | 19 | ~~20~~ | |

# Prime Number Generation

**The Critical Optimization:** The √n Limit

**The Inefficiency:** Did you notice that when we got to p = 5 in our last example, all of its multiples (10, 15, 20) were already crossed out by 2 and 3?

**The Rule:** You do NOT need to run the loop all the way to n. You only need to run the loop while p ≤ √n.

| | | | | |
|---|---|---|---|---|
| 2 | 3 | ~~4~~ | 5 | ~~6~~ |
| 7 | ~~8~~ | ~~9~~ | ~~10~~ | 11 |
| ~~12~~ | 13 | ~~14~~ | ~~15~~ | ~~16~~ |
| 17 | ~~18~~ | 19 | ~~20~~ | |

**Applying the Rule to n = 20:**

- √20 ≈ 4.47
- We only need to check primes up to 4 (which are just 2 and 3).
- Once we finished crossing out multiples of 3, the algorithm was already done!

# Prime Numbers

## Primality Testing: The Trial Division Algorithm

```
input n
output d

is_prime(n)
{
        for d = 2 to ⌊ √n ⌋
          if ( n mod d == 0 )
                return d
        return 0
}
```

**The Algorithm Breakdown**:

- Input: An integer n > 1.
- Output: Returns 0 if n is prime. If n is composite, it returns the smallest divisor d.

**The Floor Function:** ⌊x⌊ represents the greatest integer less than or equal to x.

# Prime Numbers

## Primality Testing: The Trial Division Algorithm

```
input n
output d

is_prime(n)
{
        for d = 2 to ⌊ √n ⌋
          if ( n mod d == 0 )
                return d
        return 0
}
```

- If a number n is **composite**, it must have two factors: **a × b** = n.
- If both factors were strictly greater than √n, their product would be greater than n, which is impossible.
- Therefore, the smallest prime factor must be ≤ √n.
- **Efficiency**: This optimization reduces the **Time Complexity from O(n) to O(√n)**, saving massive amounts of processing time for large numbers.

# Prime Numbers

## Primality Testing: The Trial Division Algorithm

```
input n
output d

is_prime(n)
{
        for d = 2 to ⌊ √n ⌋
          if ( n mod d == 0 )
                return d
        return 0
}
```

**Test Case A: n = 43**

- √43 ≈ 6.55. (Floor is 6).
- Test divisors d = 2 to 6.
- None divide 43 evenly. Return 0 (Prime).

**Test Case B: n = 451**

- √451 ≈ 21.23. (Floor is 21).
- Test divisors d = 2 to 21.
- When d = 11, (451 mod 11 == 0). Return 11 (Composite).

# The Fundamental Theorem of Arithmetic

## The "Digital DNA" of Integers

Every integer greater than 1 is either a prime number itself, or it can be written as a product of prime numbers in exactly one unique way.

**For any integer n > 1:**

$$n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$$

*(Where $p_1$, $p_2$, etc., are distinct prime numbers, and $a_1$, $a_2$, etc., are positive integers).*

# The Fundamental Theorem of Arithmetic

**Example:** 1200

1. Divide by 2: 1200 → 600 → 300 → 150 → 75 (That's four 2s: $2^4$)
2. Divide by 3: 75 → 25 (That's one 3: $3^1$)
3. Divide by 5: 25 → 5 → 1 (That's two 5s: $5^2$)

**Final Unique Factorization:** $1200 = 2^4 \times 3^1 \times 5^2$

**NOTE:** Because factorization is unique, computers can store massive numbers simply as a list or hash map of their prime exponents! (e.g., storing [4, 1, 2] for the primes 2, 3, and 5 is much more memory-efficient in specific algorithms).

# TRY

**Instructions:** Find the unique prime factorization for each of the following numbers. Express your answer using exponents (e.g. $2^3 \cdot 3^1 \cdot 5^1$)

1. Number: 1080
2. Number: 404
3. Number: 3600
4. Number: 1001
5. Number: 1337

# ANSWER KEY

1. $1080 = 2^3 \cdot 3^3 \cdot 5^1$
2. $404 = 2^2 \cdot 101^1$
3. $3600 = 2^4 \cdot 3^2 \cdot 5^2$
4. $1001 = 7^1 \cdot 11^1 \cdot 13^1$
5. $1337 = 7^1 \cdot 191^1$

**NOTE:** The last number is the "boss level" of the drill. You will have to test primes up to $\sqrt{1337} \approx 36$. Once you test 7, you will get $1337 \div 7 = 191$. Then, you can check primes up to $\sqrt{191} \approx 13$ to prove 191 is prime.

# Module 1 Recap:

- *Subtopic 1: We learned the syntax and data types (Real numbers, Integers, Summations).*
- *Subtopic 2: We built the logic engine and learned how to prove our code works (Peano, Induction).*
- *Subtopic 3: We discovered the "Digital DNA" of numbers—Primes.*

***TODO**: Answer Formative Assessment 1*

# Number Theory

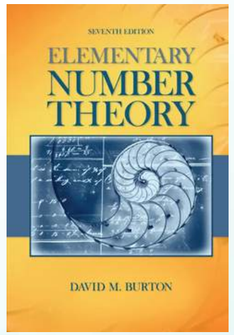
(CCS0047)

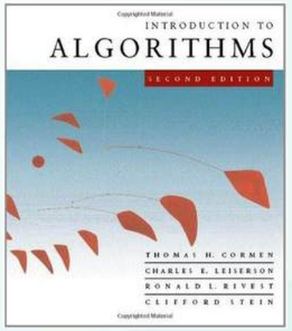
End of Module 1

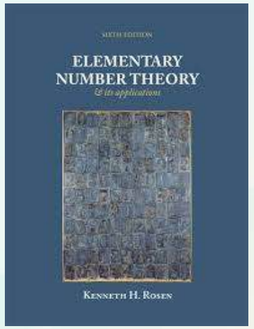
**Next Module → Module 2: The Algorithmic Integer**

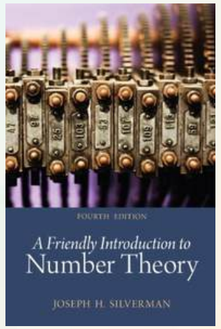
# References

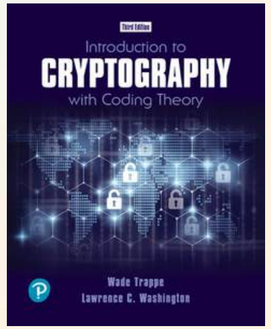

- Burton, D. M. (2011). Elementary number theory (7th ed.). McGraw-Hill Education.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.
- Rosen, K. H. (2011). Elementary number theory and its applications (6th ed.). Pearson.
- Silverman, J. H. (2013). A friendly introduction to number theory (4th ed.). Pearson.
- Trappe, W., & Washington, L. C. (2020). Introduction to cryptography with coding theory (3rd ed.). Pearson.


*Updated as of April 2026*